BeaconKit 아키텍처 분석 - 04. 블록 생성 프로세스

1. 블록 생성 개요

BeaconKit의 블록 생성은 **CometBFT의 PrepareProposal** 단계에서 시작되며, **ValidatorService의 BuildBlockAndSidecars** 메서드를 통해 수행됩니다. 이 프로세스는 실행 레이어와의 긴밀한 협력을 통해 이더리움 호환 블록을 생성합니다.

1.1 핵심 흐름

```
CometBFT PrepareProposal → ValidatorService.BuildBlockAndSidecars → 실행 엔진 페이
```

1.2 성능 목표

- **전체 프로세스**: ~1초 내 완료
- **페이로드 빌딩**: 병렬 처리로 최적화
- **상태 계산**: 캐시 활용으로 최적화

2. BuildBlockAndSidecars 메서드 상세 분석

2.1 메서드 시그니처

```
func (s *Service) BuildBlockAndSidecars(
ctx context.Context,
```

```
slotData *types.SlotData,
) ([]byte, []byte, error)
```

입력:

- `ctx` : 컨텍스트 (타임아웃 및 취소 처리)
- `slotData` : 슬롯 정보 (높이, 제안자, 합의 시간)

출력:

- `[]byte` : 서명된 비콘 블록 (SSZ 인코딩)
- `[]byte` : 블롭 사이드카들 (SSZ 인코딩)
- `error` : 오류 정보

2.2 전체 프로세스 단계

```
func (s *Service) BuildBlockAndSidecars(
ctx context.Context,
slotData *types.SlotData,
) ([]byte, []byte, error) {
startTime := time.Now()
defer s.metrics.measureRequestBlockForProposalTime(startTime)

// STEP 1: 페이로드 빌더 활성화 확인
if !s.localPayloadBuilder.Enabled() {
return nil, nil, builder.ErrPayloadBuilderDisabled
}
```

```go
// STEP 2: 상태 준비
st := s.sb.StateFromContext(ctx)
blkSlot := slotData.GetSlot()
// STEP 3: 슬롯 처리 (상태를 요청된 슬롯까지 진행)
if _, err := s.stateProcessor.ProcessSlots(st, blkSlot); err != nil {
return nil, nil, err
}

// STEP 4: 부모 블록 루트 획득
parentBlockRoot, err := st.GetBlockRootAtIndex(
(blkSlot.Unwrap() - 1) % s.chainSpec.SlotsPerHistoricalRoot(),
)

// STEP 5: 실행 페이로드 획득
envelope, err := s.retrieveExecutionPayload(ctx, st, parentBlockRoot, slotData)

// STEP 6: 포크 데이터 빌드
timestamp := envelope.GetExecutionPayload().GetTimestamp()
forkData, err := s.buildForkData(st, timestamp)

// STEP 7: 빈 블록 생성
blk, err := s.getEmptyBeaconBlockForSlot(st, blkSlot, forkData.CurrentVersion, p
```

```go
// STEP 8: RANDAO 리빌 생성
reveal, err := s.buildRandaoReveal(forkData, blkSlot)

// STEP 9: 블록 바디 조립
if  err = s.buildBlockBody(ctx, st, blk, reveal, envelope);  err != nil {
return  nil, nil, fmt.Errorf("failed build block body: %w", err)
}

// STEP 10: 상태 루트 계산 및 설정
if  err = s.computeAndSetStateRoot(ctx, slotData.GetProposerAddress(), slotData.
return  nil, nil, err
}

// STEP 11: 블록 서명
signedBlk, err := ctypes.NewSignedBeaconBlock(blk, forkData, s.chainSpec, s.sign

// STEP 12: 블롭 사이드카 생성
sidecars, err := s.blobFactory.BuildSidecars(signedBlk, envelope.GetBlobsBundle(

// STEP 13: SSZ 직렬화 및 반환
signedBlkBytes, _ := signedBlk.MarshalSSZ()
sidecarsBytes, _ := sidecars.MarshalSSZ()
```

```
return signedBlkBytes, sidecarsBytes, nil
}
```

3. 실행 페이로드 획득 프로세스

3.1 retrieveExecutionPayload 메서드

이 메서드는 블록 생성의 핵심으로, 실행 레이어와의 협력을 통해 페이로드를 획득합니다.

```
func (s *Service) retrieveExecutionPayload(
ctx context.Context,
st *statedb.StateDB,
parentBlockRoot common.Root,
slotData *types.SlotData,
) (ctypes.BuiltExecutionPayloadEnv, error) {
slot := slotData.GetSlot()
// STEP 1: 캐시된 페이로드 시도
envelope, err := s.localPayloadBuilder.RetrievePayload(ctx, slot, parentBlockRoo
if err == nil {
return envelope, nil
}

// STEP 2: 동기 페이로드 요청 (캐시 실패 시)
s.metrics.failedToRetrievePayload(slot, err)
```

```
// ... 페이로드 속성 준비 ...

return  s.localPayloadBuilder.RequestPayloadSync(ctx, r)

}
```

3.2 페이로드 빌딩 전략

1. 캐시된 페이로드 우선 사용:

- 이전에 `forkchoiceUpdate` 로 요청한 페이로드 ID 사용
- 빠른 응답 시간 보장

2. 동기 페이로드 요청 (폴백):

- 캐시 실패 시 즉시 새 페이로드 요청
- 더 긴 대기 시간이지만 확실한 페이로드 획득

3.3 페이로드 속성 구성

```
r := &builder.RequestPayloadData{

Slot: slot, // 현재 슬롯

Timestamp: nextPayloadTimestamp, // 페이로드 타임스탬프

PayloadWithdrawals: payloadWithdrawals, // 예상 출금들

PrevRandao: prevRandao, // 이전 RANDAO

ParentBlockRoot: parentBlockRoot, // 부모 블록 루트

HeadEth1BlockHash: lph.GetBlockHash(), // 현재 실행 헤드

FinalEth1BlockHash: lph.GetParentHash(), // 최종 실행 블록

ParentProposerPubkey: parentProposerPubkey, // 부모 제안자 공개키
```

```
}
```

4. 블록 바디 조립 프로세스

4.1 buildBlockBody 메서드

```
func (s *Service) buildBlockBody(
ctx context.Context,
st *statedb.StateDB,
blk *ctypes.BeaconBlock,
reveal crypto.BLSSignature,
envelope ctypes.BuiltExecutionPayloadEnv,
) error {
body := blk.GetBody()
// STEP 1: RANDAO 리빌 설정
body.SetRandaoReveal(reveal)
// STEP 2: 블롭 KZG 커밋먼트 설정
blobsBundle := envelope.GetBlobsBundle()
body.SetBlobKzgCommitments(blobsBundle.GetCommitments())
// STEP 3: 예치 처리
depositIndex, err := st.GetEth1DepositIndex()
deposits, localDepositRoot, err := s.sb.DepositStore().GetDepositsByIndex(
ctx,
```

```
constants.FirstDepositIndex,
depositIndex+s.chainSpec.MaxDepositsPerBlock(),
)
eth1Data := ctypes.NewEth1Data(localDepositRoot)
body.SetEth1Data(eth1Data)
body.SetDeposits(deposits[depositIndex:])
// STEP 4: 그래피티 설정
sizedGraffiti := bytes.ExtendToSize([]byte(s.cfg.Graffiti), bytes.B32Size)
graffiti, _ := bytes.ToBytes32(sizedGraffiti)
body.SetGraffiti(graffiti)
// STEP 5: 동기 집계 설정 (빈 값)
body.SetSyncAggregate(&ctypes.SyncAggregate{})
// STEP 6: 실행 페이로드 설정
body.SetExecutionPayload(envelope.GetExecutionPayload())
// STEP 7: 실행 요청 설정 (Electra 포크 이후)
if version.EqualsOrIsAfter(body.GetForkVersion(), version.Electra()) {
encodedReqs := envelope.GetEncodedExecutionRequests()
requests, err := ctypes.DecodeExecutionRequests(encodedReqs)
body.SetExecutionRequests(requests)
}
return nil
}
```

4.2 블록 바디 구성 요소

1. RANDAO Reveal:

- 현재 슬롯의 에포크에 대한 BLS 서명
- 랜덤성 생성에 사용

2. 블롭 KZG 커밋먼트:

- 실행 페이로드의 블롭 트랜잭션에서 추출
- 데이터 가용성 증명에 사용

3. 예치 (Deposits):

- 실행 레이어의 예치 계약에서 동기화
- 새로운 검증자 활성화에 사용

4. Eth1Data:

- 예치 머클 트리 루트
- 실행 레이어와의 동기화 증명

5. 실행 페이로드:

- 실행 레이어에서 빌드된 블록
- 트랜잭션, 상태 루트, 영수증 루트 포함

6. 실행 요청 (Electra+):

- 검증자 예치, 출금, 통합 요청
- 포크 업그레이드에서 도입

5. 상태 루트 계산

5.1 computeAndSetStateRoot 메서드

```go
func (s *Service) computeAndSetStateRoot(
ctx context.Context,
proposerAddress []byte,
consensusTime math.U64,
st *statedb.StateDB,
blk *ctypes.BeaconBlock,
) error {
stateRoot, err := s.computeStateRoot(ctx, proposerAddress, consensusTime, st, bl
if  err != nil {
return  err
}
blk.SetStateRoot(stateRoot)
return  nil
}
```

5.2 상태 전환 실행

```go
func (s *Service) computeStateRoot(
ctx context.Context,
proposerAddress []byte,
consensusTime math.U64,
st *statedb.StateDB,
```

```
blk *ctypes.BeaconBlock,
) (common.Root, error) {
// 전환 컨텍스트 생성 (검증 비활성화)
txCtx := transition.NewTransitionCtx(ctx, consensusTime, proposerAddress).
WithVerifyPayload(false). // 페이로드 검증 스킵
WithVerifyRandao(false). // RANDAO 검증 스킵
WithVerifyResult(false). // 결과 검증 스킵
WithMeterGas(false) // 가스 측정 스킵


// 상태 전환 실행
if _, err := s.stateProcessor.Transition(txCtx, st, blk); err != nil {
return common.Root{}, err
}


// 상태 루트 반환
return st.HashTreeRoot(), nil
}
```

최적화 포인트:

- 블록 빌딩 시에는 검증을 스킵하여 성능 향상
- 실제 검증은 ProcessProposal에서 수행

6. 블록 서명 및 사이드카 생성

6.1 블록 서명

```
signedBlk, err := ctypes.NewSignedBeaconBlock(blk, forkData, s.chainSpec, s.sign
```

서명 프로세스:

1. 블록의 서명 루트 계산
2. 포크 데이터와 도메인 사용
3. BLS 서명 생성
4. SignedBeaconBlock 구조체 생성

6.2 블롭 사이드카 생성

```
sidecars, err := s.blobFactory.BuildSidecars(signedBlk, envelope.GetBlobsBundle(
```

사이드카 구성 요소:

- **블롭 데이터**: 실제 블롭 내용
- **KZG 커밋먼트**: 블롭에 대한 다항식 커밋먼트
- **KZG 증명**: 커밋먼트 정확성 증명
- **포함 증명**: 블록에 포함되었음을 증명하는 머클 증명

7. 성능 최적화 전략

7.1 병렬 처리

```
// TODO: We can optimize to pre-compute this in parallel?
```

```
reveal, err := s.buildRandaoReveal(forkData, blkSlot)
```

최적화 기회:

- RANDAO 리빌 사전 계산
- 블롭 KZG 증명 병렬 생성
- 예치 데이터 백그라운드 동기화

7.2 캐싱 전략

1. 페이로드 캐싱:

- `forkchoiceUpdate` 응답 캐시
- 빠른 페이로드 검색

2. 상태 캐싱:

- 계산된 상태 루트 캐시
- 중복 계산 방지

7.3 메트릭 및 모니터링

```
startTime := time.Now()
defer  s.metrics.measureRequestBlockForProposalTime(startTime)
defer  s.metrics.measureStateRootComputationTime(startTime)
```

추적 메트릭:

- 전체 블록 빌딩 시간
- 페이로드 검색 시간

- 상태 루트 계산 시간
- 실패율 및 재시도 횟수

8. 오류 처리 및 복구

8.1 페이로드 빌딩 실패

```
envelope, err := s.localPayloadBuilder.RetrievePayload(ctx, slot, parentBlockRoo
if err == nil {
return envelope, nil
}

// 캐시 실패 시 동기 요청으로 폴백
s.metrics.failedToRetrievePayload(slot, err)
return s.localPayloadBuilder.RequestPayloadSync(ctx, r)
```

8.2 상태 전환 실패

```
if _, err := s.stateProcessor.Transition(txCtx, st, blk); err != nil {
return common.Root{}, err
}
```

실패 시나리오:

- 잘못된 블록 구조

- 상태 불일치
- 실행 페이로드 오류

8.3 타임아웃 처리

```
ctx  context.Context  // 컨텍스트를 통한 타임아웃 관리
```

타임아웃 전략:

- 전체 프로세스 타임아웃
- 실행 엔진 호출 타임아웃
- 단계별 부분 타임아웃

9. 설계 원칙

9.1 결정론적 빌딩

- 동일한 입력에 대해 동일한 블록 생성
- 비결정적 요소 제거

9.2 성능 우선

- 1초 내 완료 목표
- 병렬 처리 최대 활용
- 불필요한 검증 스킵

9.3 안전성 보장

- 실패 시 안전한 폴백

- 부분 실패에 대한 복구
- 컨텍스트 취소 시 즉시 중단

이 블록 생성 프로세스는 BeaconKit이 이더리움 호환 블록을 효율적으로 생성하면서도 CometBFT의 합의 타이밍 요구사항을 만족시키는 핵심 메커니즘입니다. 다음 문서에서는 생성된 블록의 처리 및 검증 과정을 살펴보겠습니다.